CS482/495/496 Software Project: Texas Hold'em Poker Game

Alejandro Escorcia, Bonita Rodrigues, Camila Cosme, Gigi Li

2026-10-26

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

- Client name:
- Client title:
- Client email address: lcui@loyola.edu
- Client employer:
- How you know the client:

2 Project Description

2.1 Overview

[Add a few paragraphs describing your project succinctly. What problem are you trying to solve, what is the purpose of your project? Why does your client want this project?]

2.2 Key Features

[At this point you should have a basic understanding of your client's needs. List out the key features of the software system the client wants you to build.]

2.3 Why this Project is Interesting

[Why did you decide this project was interesting enough to you to be a capstone project? What about this project is enticing? Why should anyone care?]

2.4 Areas of CS required

[What subfields of computer science seem most likely to be relevant to your project? A capstone must involve multiple.]

2.5 Potential Concerns and Questions

[Is there any aspect of this project that makes you unsure if it will work, either due to your own interests/background, or that you aren't sure if it fits the requirements? Are there questions you have about this project that you want instructor feedback about?]

2.6 Summary of Efforts to Find a Project

(Not necessary for 482) [Briefly list out when/how you've discussed with this client, and if you've discussed with other clients who either didn't work out or didn't respond. If you considered a different project and it didn't work out, why didn't it work out?]

[Most CS495 projects end here. The sections below are for CS482 and CS496 software projects].

2.7 Comparison to Draft

[For CS496 only, focus on highlighting the major differences between the draft proposal in CS495 and this one here. If there are no major differences, you can remove this subsection.]

3 Requirements

3.1 Non-Functional Requirements

[Non-functional requirements are just as important as functional requirements. Dont forget to specify them.]

Table 1 presents the NFRs for this software project.

ID	NFR Title	Category	Description
NFR1	NFR Example 1	Usability	Description of the NFR (it does not follow a user story template)
NFR2	NFR Example 2	Security	Description of the NFR (it does not follow a user story template)

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

[In CS482, all functional requirements are written as User Stories. In CS496, some projects may use a different template to write the requirements. The table below is an example of writing the Stories. Adapt accordingly to different templates or if you want to display more info.]

Table 2 presents the functional requirements for this software project.

ID	Story Title	Points	Description
S1	Story Example 1	5	As a user, I want to write a user story example, so that people will understand them.
S2	Story Example 2	2	As a user, I want to write a user story example, so that people will understand them.
	Total:	80	

Table 2: Functional requirements as User Stories.

3.3 AI Analysis on Functional Requirements

[After you receive your revised User Stories in CS482, or after your first delivery of Stories on CS496, you should use AI as a senior software engineer to help you find any missing or hidden requirements.] [You must provide which LLM you use, which prompt you used, and most importantly *reflect* on the results the LLM gave back. Just because the LLM suggested something, it does not mean it was a valid suggestion and that you should follow blindly. Use your knowledge and reason on which ones are reasonable to add to your requirements and which ones are not.]

LLM of Choice: Gemini
Prompt: Hello, these are our user stories right now can you take a look at them and see what we might be missing (for all new stories created put them in a table in latex format with title, points, priority, and story) *user stories attached to the prompt*

ID	Title	Points	Priority	User Story
PG12	Side Pot Handling	3	High	As a Player, I want side pots calculated automatically during multi-way all-ins, so that chip distributions are fair based on stack sizes.
PE23	Block / Mute Player	2	Medium	As a Player, I want to block or mute specific users, so that I don't see their chat messages or receive unwanted friend requests.
PE24	Private Table Creation	2	Medium	As a Player, I want to set a password when creating a table, so that I can play exclusively with my friends.
A6	Manage Active Tables	3	High	As an Admin, I want to view, freeze, or terminate active game tables, so that I can resolve table glitches or remove abusive players.

Table 3: Proposed New User Stories for Missing Gameplay and Administrative Features

Of all of the provided stories I think that PG12 and PER24 is valid. PG12 is definitely needed for gameplay so that we can ensure that pots are split in the case that one player goes all in and the other players decide to raise. PE24 is also needed so that users have the option to play with just friends but it is not too important. For the time being as long as players can play with each other that is enough (so medium/low priority makes sense). For PE23 it would be a nice feature to have but Im thinking it would be a feature thats very low priority so we don't think its worth implmenenting (unless we have extra time for it). For A6 again it would be a nice feature to have but our client only asked for a report feature when it came to users. Terminating active tables would be more time intensive as well.

4 System Design

4.1 Architecture

Our team is following a layered client-server architecture. We are using HTML, CSS, and JavaScript for the front-end, Node.js and Express.js for the backend, and MongoDB for data storage. The front-end communicates with the backend through Restful APIs, which handle requests and application logic.

4.2 Diagrams

4.3 Technology

Since we are using the MERN stack, most of our code will be written in JavaScript. Another technology that we will use extensively is HTML with React to provide a highly usable and aesthetically pleasing interface that merges well with our backend. Some other libraries we will use are bycrpyt (for security) and bootstraps (for HTML styling). We intend on using Jest to easily test all of our code. Finally, for our database, we plan to use MongoDB since it will be the most cost effective for our small scale project.

4.4 Coding Standards

We will practice a hybrid of object oriented programming and functional programming for most of our code base. We are using standard formatting with the Pascal case for classes and the camel case for other variables and functions. Our naming convention for modules is: [ObjectName].js, for example, 'Player.js'. We require of ourselves a minimum 80% coverage in our tests to mitigate future issues.

4.5 Data

We will have two main tables in our database. The first will store relevant information for accounts such as username, password, privileges, points, game history, badges, and friends list. The table in our database is for transactions and will log any movement of funds within our application. We will store information on active tables on the server side since most of that data needs to be readily accessible.

4.6 UI Mocks

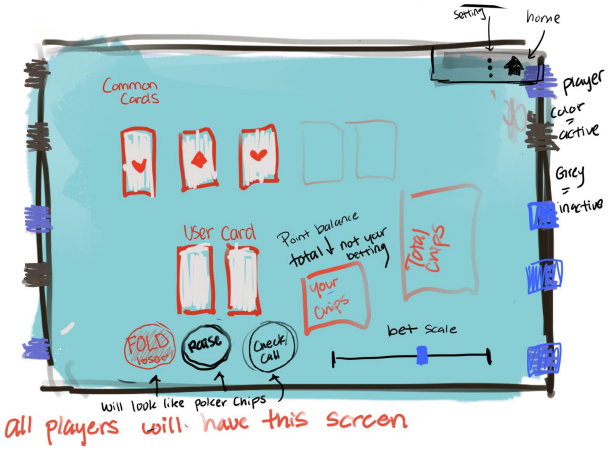


Figure 1: In-game session interface showing dealt community cards, active/inactive player status, the pot, player chips, and bet controls including Fold, Raise, and Check/Call chips.

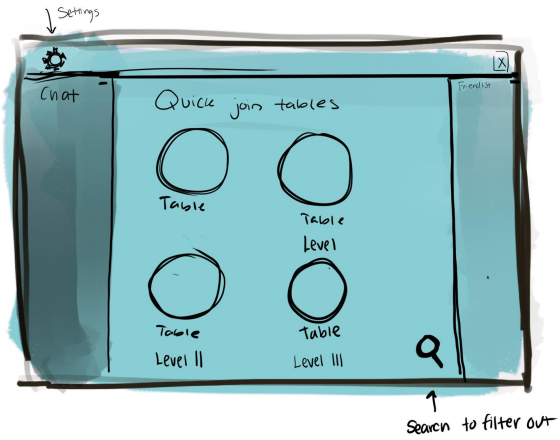


Figure 2: Lobby UI displaying quick join tables separated by skill levels (beginner, intermediate, advanced), search filters, global chat, and online friends presence tracking.

5 Iterations

5.1 Iteration Planning

[In CS496, you plan all iterations beforehand. In CS482, you update the planning here at each iteration.]

Table 4 shows the iteration planning.

It.	Dates	Stories	Points Planned	Points Done
1	10/07 - 10/22	PG1 See Cards, PG2 Bet Blindly, PG3 Raise/Check/Call/Fold, PG6 Flop, PG7 Turn, PG8 River, and PG10 Select Winner, PE7 Join Table, PG12 Set Time Limit, and PG13 Leave Game, A1 Manage Accounts, A5/PE3 Login, PE12 Create Account PE5 Friend Request, PE19 FriendList	24	24
2	10/22 - 11/05	S0 Story Title	00	00
3	11/05 - 11/19	S0 Story Title	00	00
4	date	S0 Story Title	00	00
5	date	S0 Story Title	00	00
		Total:	24	24

Table 4: Iteration Planning for Incremental Deliveries

5.2 Iteration/Sprint 1

5.2.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]
Our team is focusing on developing a functional Texas Hold'Em poker game that allows players to log in, join a table, participate in a game, and leave when they are finished. We selected these user stories to prioritize the core gameplay mechanics while also implementing the basic account management and multiplayer features needed to make the game usable. Since our selected user stories total 20 points, we are aiming to deliver a focused set of connected features rather than attempting to implement every possible poker feature at once.

The main focus of our project is to make sure players can complete the main stages of a Texas Hold'Em game. PG1 (See Cards) allows players to view their cards, while PG2 (Bet Blindly) establishes the initial bets needed to start a hand. PG3 (Raise/Check/Call/Fold) allows players to make decisions during betting rounds. PG6 (Flop), PG7 (Turn), and PG8 (River) cover the stages of dealing the community cards. Finally, PG10 (Select Winner) determines the winner based on the players' hands. These features depend on one another, so implementing them will allow us to create a complete gameplay experience, from the beginning of a hand to determining the winner. We want to prioritize these mechanics because they are necessary for the game to function as an actual poker game rather than just a website where players can join a table.

In addition to the core gameplay, we are focusing on making the game accessible and manageable for players. PE7 (Join Table) allows players to enter a game, while PG13 (Leave Game) allows them to leave when they no longer wish to participate. These features require the game to keep track of which players are at each table and update the table's state when players join or leave. PG12 (Set Time Limit) adds a time constraint to player actions, helping the game progress without unnecessary delays when players take too long to make decisions. Together, these features support a smoother multiplayer experience and help prevent the game from becoming stalled.

We will also implement A1 (Manage Accounts), A5/PE3 (User and Admin Login) and PE12 (Create User Account) provide the basic account functionality needed for players to access the game. Login allows the system to identify players, while account management supports the organization and maintenance of player information. These features provide a foundation for managing players and connecting them to their game

sessions. Building on this authentication and profile management, we also integrated PE5 (Friend Request) and PE19 (Friendlist) to enrich the multiplayer ecosystem. These features enable players to search for other users by username handle, manage pending incoming requests, and maintain an active friends list, bridging account identity with interactive community gameplay.

5.2.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

- Camila: PG3 (Raise/Check/Call)
    1. PG1 (See Cards) — Complete: (2 hours) - completed: In the backend the show cards belonged in the pokerlogic
    2. PG2 (Bet Blindly) — Complete : Blinds, 1 point (4 hours) - completed: In the backend I realized I needed to create methods for the dealer which took a while to implement. Based on the dealer the small blind and big blind were created with the assignBlinds function. Then the postBlind helper function and postBlinds function handles adding money to the pot. It took me a bit to realize we have three different table values for blinds. Honestly I was pretty happy with this method though it took a long time. I think I would want to clean the logic for the table functions.
    3. PG3 (Raise/Check/Call) — Complete: (5 hours) - completed: In the backend the methods raise, check, fold, and call were created. On the front end the methods were raisePlayer, checkPlayer, foldPlayer, and callPlayer which are all connected to the front end. These methods are added to the app.js. Honestly if I had to redo this I would want to refactor it and make it more clean because this got really messy really fast.
- Alejandro:
    1. PG6 (Flop), PG7 (Turn), PG8 (River) & Core Deck Utility — Complete: (6 hours) - completed: Designed and implemented the `Deck` utility class in `pokerLogic.js` featuring secure Fisher-Yates shuffling and precise card dealing for player hole cards, flop, turn, and river streets. Developed the `ActiveTable` backend and controller logic to handle community card progression, betting round completion, and bet-matching validation.
    2. PG10 (Select Winner) & End-of-Game Logic — Complete: (5 hours) - completed: Implemented robust poker hand evaluation algorithms and rank hierarchies (from High Card to Royal Flush) using 5-card combinations via `Deck.bestHand`. Integrated showdown detection that automatically computes pot shares, distributes chips to winners, and triggers lobby cleanup.
    3. Gameplay Infrastructure & Refactoring — Complete: (8 hours) - completed: Refactored the entire gameplay codebase (`activeTable.js`, `tableController.js`, and frontend views) into a maintainable, robust architecture. Built turn-based state management with button-blocking controls, a dealer tracker (D) rotating between games, aggressor tracking with update-on-raise/fold rules, small/big blind posting at game start, 30-second turn timers with automatic check/fold upon timeout, and deferred player removal/kick mechanisms to handle disconnections and mid-game joins cleanly without locking the table.
- Gigi:
    1. PE7 (Join Table) — (4 hours) Complete: Implemented the backend functionality for joining a table and adding players. I developed `tableController.js` and `ActiveTable.js`, along with their respective tests, to support table operations and player management. I also developed the poker table frontend in `tableDisplay.html` (3 hours), including the table layout, player display, and a consistent visual theme. My teammates built on the frontend I created by implementing gameplay features such as turn progression and player actions, including raising, checking, folding, and calling. I also worked on testing and debugging the integration between the frontend and backend.

2. PG12 (Set Time Limit) — (7 hours) Partially Complete: Implemented parts of the timer functionality in `ActiveTable.js`, including tracking the remaining time, checking whether a player's turn had expired, and automatically folding a player when the timer ran out. I also wrote tests for the table functionality. Additional integration was needed to coordinate the timer with turn progression and player actions such as folding, checking, raising, and calling.
3. PG13 (Leave Game) — (4 hours) Complete**:** Implemented functionality for removing players from a table and updating the table state. I worked on the player-removal logic in `ActiveTable.js` and the related controller functionality in `tableController.js`, along with their respective tests. This work focused on ensuring that players could leave the table and that the player list and table state were updated accordingly.

- Bonita:

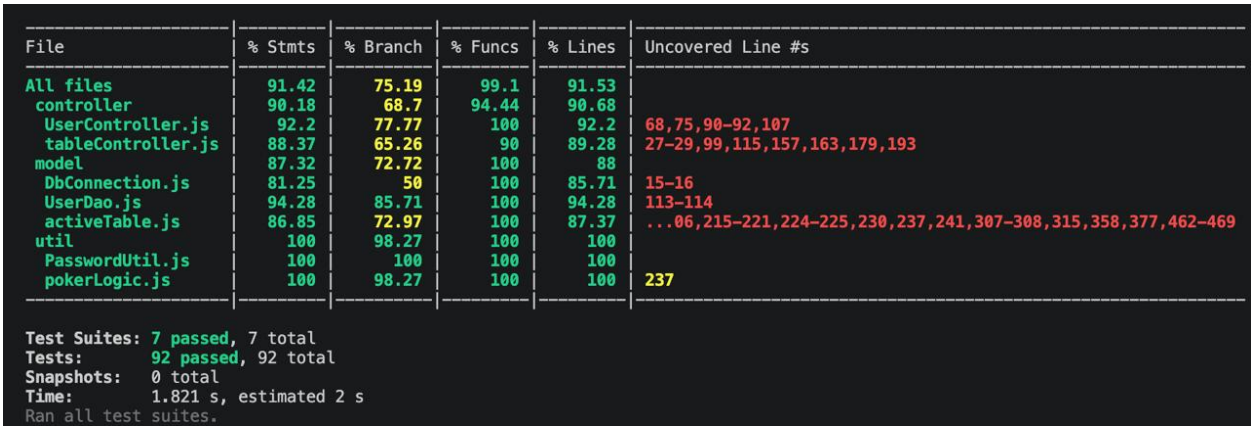
1. A1 (Manage Account) — (4hrs) Complete: Developed the administrative and user-level management system. In the backend, `UserController.postCreateOrUpdate` handles record modifications, permission assignments, and secure password re-hashing, while `getAll` and `deleteOne` enforce strict admin security checks. On the frontend, `admin.html` and `admin.js` dynamically render a responsive user table with role badges and link to a Bootstrap edit modal for adjusting points, roles, and account details.
2. A5/PE3 (User/Admin Login) —(2hrs) Complete: Implemented secure authentication and session management. The backend leverages `UserController.login` and `PasswordUtil` to hash and compare credentials, establishing persistent session objects (`req.session.user`) with role-based attributes. On the frontend, login forms communicate with these endpoints, automatically routing administrators to the admin dashboard and standard players to the main lobby.
3. PE12 (Create Account) — (1hr) Complete: Developed the user registration pipeline. In the backend, `UserController.register` processes new user signups, delegates secure password hashing to `PasswordUtil`, initializes starting chip balances, assigns default permissions, and instantiates the initial session. On the frontend, the registration form captures user credentials and submits them, automatically authenticating and routing the newly registered player into the application.
4. PE5 (Friend Request) — (5hrs) Complete: Built the backend and frontend pipeline for sending, accepting, and declining social friend requests. The backend features `UserController.searchUsers`, `sendFriendRequest`, `acceptFriendRequest`, and `declineFriendRequest`, which dynamically resolve user identifiers and update database arrays atomically. On the frontend, users can search for peers by username, view search results, and trigger friend requests or manage incoming responses directly from the interface.
5. PE19 (FriendList) — (2hrs) Complete: Implemented the friend and pending request data retrieval system. The backend `UserController.getFriendsData` endpoint uses a robust `resolveToUsername` helper to dynamically normalize legacy email identifiers into clean username handles on the fly. On the frontend, this structured JSON data populates the active friends list and pending request tabs to complete the multiplayer social experience.

Some gameplay features were developed collaboratively, with everyone building on existing frontend and table implementation.

5.2.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

Figure 3 shows the test coverage



File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	91.42	75.19	99.1	91.53	
controller	90.18	68.7	94.44	90.68	
UserController.js	92.2	77.77	100	92.2	68,75,90-92,107
tableController.js	88.37	65.26	90	89.28	27-29,99,115,157,163,179,193
model	87.32	72.72	100	88	
DbConnection.js	81.25	50	100	85.71	15-16
UserDao.js	94.28	85.71	100	94.28	113-114
activeTable.js	86.85	72.97	100	87.37	...06,215-221,224-225,230,237,241,307-308,315,358,377,462-469
util	100	98.27	100	100	
PasswordUtil.js	100	100	100	100	
pokerLogic.js	100	98.27	100	100	237

Test Suites: 7 passed, 7 total
Tests: 92 passed, 92 total
Snapshots: 0 total
Time: 1.821 s, estimated 2 s
Ran all test suites.

Figure 3: Iteration 1 test coverage report

For this iteration, our test suite is looking really solid with 7 passed test suites and all 92 tests passing successfully. Overall, we're hitting about 91.53% line coverage and 91.42% statement coverage across all files, with function coverage sitting way up at 99.1%.

I definitely think this coverage is good enough for this iteration because we've fully covered our core user management, authentication, and password utilities, and our user DAO and controller logic are both hovering around 92–94% line coverage. Some of the artifacts, like `DbConnection.js` or parts of `tableController.js` and `activeTable.js`, don't have 100% coverage because they mostly contain error-handling `catch` blocks, database connection failures, or game-state branch conditions that have not been thoroughly tested.

To push coverage even higher next time, our plan is to write more targeted unit tests that specifically hit those leftover error branches and catch blocks. But overall, having all 92 tests passing gives us a really reliable and stable baseline for our code.

5.2.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

- Bonita: This iteration had many challenges, mostly revolving around time management since it hit right during midterms week, which shortened the coding window. Despite that, our team communication and mutual support during the sprint were really good; we stayed completely aligned on what was being built and pushed. For my part, working on the user and social stories went relatively smoothly, mostly because my code was largely independent of the core poker game (which is what everyone else worked on), allowing me to develop/test my logic without getting blocked by merging conflicts. However, I definitely learned a hard lesson about planning. I forgot to check all the incoming requirements for the friend system when I first set up the user model, meaning I had to go back and refactor the database schema and controllers later to accommodate those new parts. Another thing was trying to constantly remember and format commit messages to link back to the Kanban board (it was a hassle). Moving forward, I want to make sure I map out all future feature requirements fully before writing the code, while also doing tasks earlier in the timeline to avoid being overwhemled with multiple things.
- Camila: I think the testing in this iteration was good enough. We did get branch coverage and line coverage. I think a way to improve the process for future iteration is to work on it earlier, set different times to do the code, and communicate better with teammates. I know that I got quite confused with front end aspect which could have been clearer.

- Alejandro: This iteration was pretty intense because I has to implement poker gameplay logic, handling the deck utility and community cards (flop, turn, river) to hand evaluation for selecting the winner and turn-based system. The biggest problem for me was dealing with architectural mismatches. My core logic and state management approach didn't match up with how other parts of the system were implemented, so I had to spend a massive chunk of time completely refactoring `activeTable.js`, `tableController.js`, and the frontend views just to bridge the gap and make everything actually line up. Moving forward, I learned that we need much tighter alignment and communication on our backend assumptions and interface designs *before* we start coding, which will save us from having to do stressful structural rewrites mid-sprint.

5.3 Iteration/Sprint 2

5.3.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.3.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.3.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.3.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.4 Iteration/Sprint 3

5.4.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.4.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.4.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.4.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.5 Iteration/Sprint 4

[CS496 has 5 sprints. CS482 only has only 3 sprints (remove Iterations 4 and 5 from this doc if you are writing a doc for 482]

5.5.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.5.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.5.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.5.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.6 Iteration/Sprint 5

5.6.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.6.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.6.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.6.4 Retrospective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

6 Final Remarks

6.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project. Be concrete about your progress, you know how many story points your software is, how many points you completed (this shows your progress). You also how many points your team delivers at each iteration, therefore you can estimate how many more iterations it would take to finish the leftover points (show the math).]

6.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently? How can you do better work in future projects? You may write this as a team or per person (or both — if all your iterations were team reflections, then it would be better to write individual reflections here)]

Appendix

[Appendix section if needed]